Chapter 28 | Exercise solutions with Python

Learning from Tables

This notebook solves Exercises 5–8. It fits a row encoder on training data, checks how new and missing categories are represented, and compares a context-weighted probability with threshold and Brier scores.

The examples use the chapter's local NumPy code and small tables with known results. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch28_tabular"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Keep a held-out row out of fitting

A blank in the days column needs a replacement value. The encoder learns that value and the known channel names from the four training orders. Each position in `train_numeric` and `train_channels` refers to the same order. `None` marks a blank.

The observed training days are 2, 4, and 8. Their middle value is 4, so the fitted median is 4. The distinct observed channels in alphabetical order are `shop` and `web`. That order sets the positions for the two known-channel indicators. Each indicator is 1 when its condition holds and 0 otherwise.

```
from examples.ch28_tabular.model import encode_rows, fit_row_encoder

train_numeric = [2, 4, None, 8]
train_channels = ["web", "shop", None, "web"]
median, levels = fit_row_encoder(train_numeric, train_channels)

assert median == 4
assert levels == ("shop", "web")

columns = (
    "filled days",
    "days missing",
    *levels,
    "channel missing",
    "channel unknown",
)
print(f"Training median = {median:g}")
print("Output columns in order")
for position, name in enumerate(columns, start=1):
    print(f"{position}. {name}")
```

OUTPUT

```
Training median = 4
Output columns in order
1. filled days
2. days missing
3. shop
4. web
5. channel missing
6. channel unknown
```

Apply this fit to the three held-out orders. The row with a blank days value gets 4 and a 1 in `days missing`. Its channel `kiosk` was unseen during training, so `channel unknown` is 1. The row with 6 days and channel `shop` keeps 6 and activates `shop`. The row with 100 days and a blank channel keeps 100 and activates `channel missing`. Both `kiosk` and the blank channel leave the `shop` and `web` entries at 0.

```
held_out_numeric = [None, 6, 100]
held_out_channels = ["kiosk", "shop", None]
held_out_features = encode_rows(
    held_out_numeric, held_out_channels, median, levels
)
expected_held_out = [
    [4,   1, 0, 0, 0, 1],
    [6,   0, 1, 0, 0, 0],
    [100, 0, 0, 0, 1, 0],
]
np.testing.assert_array_equal(held_out_features, expected_held_out)
assert held_out_features.shape == (3, len(columns))
print(f"Held-out shape {held_out_features.shape}")
print(held_out_features)
```

```
OUTPUT
Held-out shape (3, 6)
[[  4.   1.   0.   0.   0.   1.]
 [  6.   0.   1.   0.   0.   0.]
 [100.   0.   0.   0.   1.   0.]]
```

Append the order with 12 days and channel `mobile`. Pass the same median and levels to `encode_rows`. The extra row keeps its observed value 12 and uses the existing `channel unknown` flag. The four-row result still has six columns, and its first three rows match the original output. The held-out values 100 and 12 appear in their own rows. The median used to fill the blank came from the training days $(2, 4, 8)$.

```
extended_features = encode_rows(
    held_out_numeric + [12],
    held_out_channels + ["mobile"],
    median,
    levels,
)
assert extended_features.shape == (4, len(columns))
np.testing.assert_array_equal(extended_features[:3], held_out_features)
np.testing.assert_array_equal(extended_features[3], [12, 0, 0, 0, 0, 1])
print(f"Extended held-out shape {extended_features.shape}")
print(extended_features)
```

```
OUTPUT
Extended held-out shape (4, 6)
[[  4.   1.   0.   0.   0.   1.]
 [  6.   0.   1.   0.   0.   0.]
 [100.   0.   0.   0.   1.   0.]
 [ 12.   0.   0.   0.   0.   1.]]
```

6. Test the effect of an unseen category

Use the fit from Exercise 5 and hold the days value at 6. Change only the channel. The two numerical entries should be $(6, 0)$ in every row because 6 is an observed number. The last four columns tell us which of the three channel cases applies.

```
channel_examples = ["shop", "kiosk", None]
comparison_features = encode_rows(
    [6, 6, 6], channel_examples, median, levels
)

np.testing.assert_array_equal(
    comparison_features[:, :2], [[6, 0], [6, 0], [6, 0]]
)
np.testing.assert_array_equal(
    comparison_features[:, 2:], [[1, 0, 0, 0], [0, 0, 0, 1], [0, 0, 1, 0]]
)
print("Channel   Encoded row")
for channel, row in zip(channel_examples, comparison_features):
    print(f"{str(channel):<9} {row}")
```

```
OUTPUT
Channel   Encoded row
shop      [6. 0. 1. 0. 0. 0.]
kiosk     [6. 0. 0. 0. 0. 1.]
None      [6. 0. 0. 0. 1. 0.]
```

For `shop`, the 1 is in column 3, the known `shop` indicator. For `kiosk`, it moves to column 6 because the channel is present and absent from the training vocabulary. For `None`, it moves to column 5 because the channel itself is missing. Column 4, the indicator for `web`, is 0 throughout. The numerical columns agree exactly. The distinction between an unseen channel and a blank one is carried by two separate categorical flags.

7. Trace the influence of a context row

Think of a query order whose label we want to predict. The first two context orders have positive labels. The third has a negative label, and the query receives a higher similarity score for that order. We will raise the third score and see how much influence it gains.

Softmax exponentiates the scores and divides by the sum of those exponentials. With scores $(0, 0, \ln 4)$, the exponentials are $(1, 1, 4)$, which sum to 6. The weights are $(1/6, 1/6, 4/6)$. The toy predictor multiplies each weight by its row's label and adds the products. Only the first two labels are 1, so the initial positive-class estimate is

$$\widehat{p} = \frac{1}{6}(1) + \frac{1}{6}(1) + \frac{4}{6}(0) = \frac{1}{3}.$$

Raising the last score by $\ln 2$ makes it $\ln 8$. Its exponential doubles from 4 to 8, while the first two stay at 1. The new denominator is 10, so the weights are $(1/10, 1/10, 8/10)$. The estimate becomes $1/10 + 1/10 = 1/5$. The table below checks both predictions and all six weights against these hand calculations.

```
import math

from examples.ch28_tabular.model import context_probability

context_labels = [1, 1, 0]
original_scores = np.array([0.0, 0.0, math.log(4)])
raised_scores = original_scores.copy()
raised_scores[-1] += math.log(2)

original_p, original_weights = context_probability(
    original_scores, context_labels
)
raised_p, raised_weights = context_probability(
    raised_scores, context_labels
)

np.testing.assert_allclose(original_weights, [1 / 6, 1 / 6, 4 / 6])
np.testing.assert_allclose(raised_weights, [1 / 10, 1 / 10, 8 / 10])
np.testing.assert_allclose([original_p, raised_p], [1 / 3, 1 / 5])
np.testing.assert_allclose(raised_p - original_p, -2 / 15)

print("Setting          Row 1     Row 2     Row 3   P(label = 1)")
for setting, weights, probability in [
    ("Original", original_weights, original_p),
    ("Raised row 3", raised_weights, raised_p),
]:
    weights_text = "  ".join(f"{weight:8.5f}" for weight in weights)
    print(f"{setting:13s}  {weights_text}  {probability:11.5f}")
```

OUTPUT

```
Setting          Row 1     Row 2     Row 3   P(label = 1)
Original       0.16667   0.16667   0.66667      0.33333
Raised row 3   0.10000   0.10000   0.80000      0.20000
```

The negative row takes 8/10 of the weight after the change, compared with 4/6 before. Because the three weights must sum to one, less weight remains for the two positive rows. The negative row's extra weight contributes zero to the weighted label sum, and the estimate drops from about 0.33333 to 0.20000. A larger similarity means more influence for that context row. The direction of the probability change depends on its label.

8. Score two confidence levels

Suppose two predictors classify the same four orders. Each gives a probability of label 1. Let A give $(0.2, 0.7, 0.8, 0.1)$ and B give $(0.45, 0.55, 0.6, 0.4)$. The observed labels are $(0, 1, 1, 0)$.

At threshold 0.5, assign label 1 when $p_i \geq 0.5$ and label 0 otherwise. Each predictor gives $(0, 1, 1, 0)$, so both get all four decisions right and achieve accuracy $4/4 = 1$.

In the chapter's binary convention, each case contributes the squared difference between its positive-class probability p_i and its observed label y_i. The Brier score is the mean of those contributions.

$$Brier = \frac{1}{4}\sum_{i=1}^{4}(p_i - y_i)^2.$$

Case	Label	A probability	A squared error	B probability	B squared error
1	0	0.20	0.0400	0.45	0.2025
2	1	0.70	0.0900	0.55	0.2025
3	1	0.80	0.0400	0.60	0.1600
4	0	0.10	0.0100	0.40	0.1600

For example, the second case has label 1. A assigns it 0.7, giving $(0.7 - 1)^2 = 0.09$. B assigns it 0.55, giving $(0.55 - 1)^2 = 0.2025$. The sums are 0.18 for A and 0.725 for B, so the scores are 0.045 and 0.18125 respectively.

```
from examples.ch28_tabular.model import brier_score

observed_labels = np.array([0, 1, 1, 0])
probability_rows = [
    ("A", np.array([0.2, 0.7, 0.8, 0.1])),
    ("B", np.array([0.45, 0.55, 0.6, 0.4])),
]
expected_errors = {
    "A": [0.04, 0.09, 0.04, 0.01],
    "B": [0.2025, 0.2025, 0.16, 0.16],
}
expected_brier = {"A": 0.045, "B": 0.18125}

for name, probabilities in probability_rows:
    decisions = (probabilities >= 0.5).astype(int)
    accuracy = float(np.mean(decisions == observed_labels))
    squared_errors = (probabilities - observed_labels) ** 2
    score = brier_score(observed_labels, probabilities)

    np.testing.assert_array_equal(decisions, [0, 1, 1, 0])
    np.testing.assert_allclose(squared_errors, expected_errors[name])
    np.testing.assert_allclose(score, expected_brier[name])
    assert accuracy == 1.0

    errors_text = ", ".join(f"{error:.4f}" for error in squared_errors)
    print(f"Predictor {name}")
    print(f"  Decisions      {decisions.tolist()}")
    print(f"  Accuracy       {accuracy:.1%}")
    print(f"  Squared errors [{errors_text}]")
    print(f"  Brier score    {score:.5f}")
```

OUTPUT

```
Predictor A
  Decisions      [0, 1, 1, 0]
  Accuracy       100.0%
  Squared errors [0.0400, 0.0900, 0.0400, 0.0100]
  Brier score    0.04500
Predictor B
  Decisions      [0, 1, 1, 0]
  Accuracy       100.0%
  Squared errors [0.2025, 0.2025, 0.1600, 0.1600]
  Brier score    0.18125
```

A and B make the same decisions, but A assigns probabilities closer to the observed labels in every case. A therefore gets the lower Brier score on these four cases. Accuracy at 0.5 gives both predictors the same credit because it only sees the decisions.

Calibration asks whether events assigned a given probability occur about that often. For example, among many independent orders that receive predictions near 0.7, the observed fraction of positive labels should be near 0.7. A single order receiving 0.7 has only one outcome, so it tells us little about that frequency.

To compare calibration, collect many held-out predictions and outcomes from the intended deployment population. Group each predictor's probabilities into the same ranges and compare the mean predicted probability with the observed positive rate in each range. Report how many orders fall in each range and attach uncertainty intervals, accounting for repeated customers if their orders are related. This can reveal where a predictor's probabilities run systematically high or low. The Brier scores here compare overall probability error on the four given cases.